

SIMATS ENGINEERING

ASSESSMENT TOOL 3

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I K. Lokesh Chandra(192425358) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lokesh Chandra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Incorrect JOIN Syntax and Duplicate Results

Problem

The query uses an incorrect JOIN condition, causing duplicate records.

Incorrect Query

```
SELECT *  
FROM Student S, Class C  
WHERE S.ClassID = S.ClassID;
```

Correct Query

```
SELECT S.StudentID,  
       S.StudentName,  
       C.ClassName  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID;
```

Explanation

- The original query compares the same column (S.ClassID = S.ClassID), creating duplicate results.
- The corrected JOIN properly matches the Student and Class tables.

2. Missing Correlation in Subquery

Incorrect Query

```
SELECT EmpName  
FROM Employee E  
WHERE Salary >  
(  
  SELECT AVG(Salary)  
  FROM Employee  
);
```

Correct Query

```
SELECT EmpName  
FROM Employee E  
WHERE Salary >  
(  
  SELECT AVG(Salary)  
  FROM Employee E2  
  WHERE E2.DeptID = E.DeptID  
);
```

Explanation

The subquery is correlated using DeptID, so each employee is compared with the average salary of their own department.

3. SQL View Error

Incorrect View

```
CREATE VIEW StudentView AS
SELECT *
FROM Student;
```

Optimized View

```
CREATE VIEW StudentView AS
SELECT StudentID,
       StudentName,
       ClassID
FROM Student;
```

Explanation

Using SELECT * in a view is not recommended because future table changes may affect the view. Selecting required columns improves clarity and performance.

4. Relational Algebra Debugging

Incorrect Expression

Student $\bowtie$ Course

Correct Expression

Student $\bowtie$ Student.CourseID = Course.CourseID Course

Explanation

A natural join should be performed on matching attributes. Specifying the join condition ensures correct output.

5. Referential Integrity Violation

Incorrect SQL

```
INSERT INTO Orders
VALUES(101,5001);
(Customer 5001 does not exist.)
```

Correct SQL

```
CREATE TABLE Customer
(
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(50)
);

CREATE TABLE Orders
(
  OrderID INT PRIMARY KEY,
```

```
CustomerID INT,  
FOREIGN KEY(CustomerID)  
REFERENCES Customer(CustomerID)  
);
```

```
INSERT INTO Customer  
VALUES(5001,'Rahul');
```

```
INSERT INTO Orders  
VALUES(101,5001);
```

Explanation

The parent record must exist before inserting child records.

6. Optimize Nested Query Using JOIN

Inefficient Query

```
SELECT StudentName  
FROM Student  
WHERE ClassID IN  
(  
SELECT ClassID  
FROM Class  
WHERE TeacherID=10  
);
```

Optimized Query

```
SELECT S.StudentName  
FROM Student S  
JOIN Class C  
ON S.ClassID=C.ClassID  
WHERE C.TeacherID=10;
```

Explanation

JOINS execute faster than nested subqueries and improve readability.

7. GROUP BY Error

Incorrect Query

```
SELECT Department,  
EmployeeName,  
SUM(Salary)  
FROM Employee  
GROUP BY Department;
```

Correct Query

```
SELECT Department,
```

```
SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Department;
```

Explanation

Every selected column must either be aggregated or included in the GROUP BY clause.

8. Incorrect UPDATE Statement

Incorrect Query

```
UPDATE Employee
SET Salary=Salary+5000;
```

Correct Query

```
UPDATE Employee
SET Salary=Salary+5000
WHERE Department='IT';
```

Explanation

Without a WHERE clause, every row is updated unintentionally.

9. Optimize Multi-table Query

Inefficient Query

```
SELECT CustomerName
FROM Customer
WHERE CustomerID IN
(
  SELECT CustomerID
  FROM Orders
  WHERE ProductID IN
  (
    SELECT ProductID
    FROM Product
    WHERE Category='Electronics'
  )
);
```

Optimized Query

```
SELECT C.CustomerName
FROM Customer C
JOIN Orders O
ON C.CustomerID=O.CustomerID
JOIN Product P
ON O.ProductID=P.ProductID
WHERE P.Category='Electronics';
```

Indexes

```
CREATE INDEX idx_customer  
ON Orders(CustomerID);
```

```
CREATE INDEX idx_product  
ON Orders(ProductID);
```

Explanation

JOINS and indexes reduce execution time and improve query performance.

10. Relational Calculus Debugging

Incorrect Expression
$$\{S \mid \forall C (\text{Enrolled}(S,C))\}$$
Correct Expression
$$\{S \mid \\ \text{Student}(S) \\ \wedge \\ \exists C \\ (\\ \text{Enrolled}(S,C) \\) \\ \}$$
Explanation

Using the universal quantifier ($\forall$) incorrectly requires a student to be enrolled in every course, often producing an empty result. Using the existential quantifier ($\exists$) correctly returns students enrolled in at least one course.